WHATSAPP BOT SHOWCASE

Show the conversation before you build the bot.

Brief an expert WhatsApp-bot agent on your business; it tells you what it understands, you correct it, and it generates a complete, animated conversation on a pixel-accurate phone. Refine either party's wording with live feedback, then export client-ready screenshots. Works offline.

Product & developer documentation · v1.2

What this is

WhatsApp Bot Showcase is a small Next.js web app that turns a JSON description of a chat into a faithful, animated WhatsApp conversation. It exists to solve a specific problem: explaining what a WhatsApp bot will *feel* like before anyone writes back-end code. Instead of static mock-ups, you hand a client a link and they watch the bot send messages, type, present buttons, open a list menu, drop a voice note, run a poll — exactly as it would on their phone.

Everything visible in the chat is generated from a single JSON object called a **Flow**. The fastest way to make one is to simply **describe a business** — “a restaurant that sells fruit”, “a dental clinic”, “a coding bootcamp” — and the tool writes a complete, on-brand conversation for it, saves it to your chats, and starts playing. You can also write Flows by hand or build them visually. Custom Flows are saved in the browser; three demos ship with the app.

At a glance

- Describe-a-business generator — type any business and get a full, themed WhatsApp conversation in one click.
- Export client-ready screenshots — a single phone screenshot, or the entire conversation as one tall image.
- Pixel-accurate WhatsApp UI in dark and light themes, designed mobile-first.
- Animated playback engine — messages reveal one by one with typing indicators, ticks and natural pacing.
- Full message vocabulary: text, image, video, voice note, document, sticker, reply buttons, list menus, polls, location, contact cards, call-to-action links, rich cards and products.
- A visual Flow Builder with a live preview, plus import / export of the JSON.
- Transport controls: play, pause, restart, skip to end and 0.5×–2× speed.

Quick start

The project is a standard Next.js 14 (App Router) app. You need Node.js 18.18+ (Node 20 LTS recommended).

Terminal

```
# 1. install dependencies
npm install

# 2. start the dev server (http://localhost:3000)
npm run dev

# 3. build for production
npm run build
npm run start
```

Open `http://localhost:3000` and you’ll land on the generator. Describe a business and press **Generate**, tap an example, or open one of the saved demos to play it. The **Builder** (top-right) is there for hand-editing and importing/exporting JSON.

Project structure

Files

```
whatsapp-bot-showcase/
├─ app/
│  ├─ page.tsx              landing — the business-idea generator
│  ├─ chat/[id]/page.tsx    player — plays a Flow + screenshot export
│  ├─ builder/page.tsx      JSON editor + live preview + generate box
│  ├─ api/generate/route.ts optional AI generation (needs an API key)
│  ├─ layout.tsx            root layout & metadata
│  └─ globals.css           theme tokens, wallpaper, bubble tails
├─ components/
│  ├─ PhoneFrame.tsx        device bezel (desktop) / full-bleed (mobile)
│  ├─ StatusBar.tsx         clock, signal, wi-fi, battery
│  ├─ PlayerControls.tsx    play / pause / restart / speed
│  ├─ ChatListScreen.tsx    the WhatsApp "Chats" screen
│  ├─ ReviewEditor.tsx      inline copy editor + live agent feedback
│  └─ chat/                 header, bubbles, renderers, list sheet,
│                           ChatExportView (static export renderer)…
├─ lib/
│  ├─ types.ts              the Flow & Message schema (source of truth)
│  ├─ generator.ts          idea/intake -> Flow (offline generator)
│  ├─ agent.ts              agent types + offline heuristics + callers
│  ├─ agentServer.ts        persona + Anthropic call (server-only)
│  ├─ useGenerator.ts       generate hook (AI route -> offline fallback)
│  ├─ capture.ts            DOM -> PNG screenshot helpers
│  ├─ usePlayer.ts          playback engine (timing, typing, reactions)
│  ├─ flows.ts              load/save, validation, previews
│  └─ format.tsx            WhatsApp markdown -> React
├─ app/api/
│  ├─ understand/           "here's what I understand" (the brief)
│  ├─ generate/             intake/brief -> Flow JSON
│  └─ analyze/              review one wording change
└─ data/flows/*.json        the three bundled demos
```

The Flow object

A Flow describes one conversation: who the bot is, how the phone looks, and the ordered list of messages. This is the entire contract — if you can produce this JSON, the app can play it.

Field	Type	Description
`id`	string	Unique slug used in the URL, e.g. "pitch-agent-demo". Auto-generated from the name on save if omitted.
`name`	string	Contact name shown in the header and chats list.
`subtitle`	string?	Header line under the name (e.g. "online", "Business Account"). Replaced by "typing…" while the bot types.
`avatar`	Avatar?	{ initials?, color?, image? }. Initials + colour, or an image URL.
`verified`	boolean?	Shows the green verified check beside the name.
`phoneTime`	string?	Clock shown in the status bar, e.g. "20:19".
`battery`	number?	Battery percentage 0–100 (turns red at 20 or below).
`theme`	"dark"|"light"	Colour scheme. Defaults to dark.
`wallpaper`	string?	"default" doodle pattern, a #hex colour, or an image URL.
`speed`	number?	Initial playback speed multiplier (1 = normal).
`messages`	Message[]	The ordered conversation. See below.

A complete Flow header

```
{
  "id": "bella-booking",
  "name": "Bella Beauty Studio",
  "subtitle": "Business Account",
  "avatar": { "initials": "BB", "color": "#c2185b" },
  "verified": true,
  "phoneTime": "18:42",
  "battery": 68,
  "theme": "dark",
  "wallpaper": "default",
  "speed": 1,
  "messages": [ /* … */ ]
}
```

Anatomy of a message

Every entry in `messages` has two independent ideas: **who sent it** and **what it contains**.

Field	Type	Description

`from`	"bot"|"user"|"system"	Side of the conversation. "bot" is incoming (left), "user" is you (right). Omit for system/date rows.
`type`	string	Content type (text, image, buttons, list…). Optional — it is inferred from whichever content field you set.
`time`	string?	Timestamp shown on the bubble, e.g. "09:41".
`status`	string?	User messages only: sending | sent | delivered | read (controls the ticks).
`quote`	Quote?	{ author, text, color? } renders a quoted reply above the bubble.
`forwarded`	boolean?	Shows the “Forwarded” label.
`starred`	boolean?	Shows a star in the meta row.
`delay`	number?	Playback: ms to wait before this message appears (overrides auto-pacing).
`typing`	boolean?	Playback: force a typing indicator before this message (or set false to suppress).

Because `type` is inferred, the shortest possible message is just a sender and a content field:

Type is inferred from the "text" field

```
{ "from": "bot", "text": "Hi! How can I help today?", "time": "09:41" }
```

Message types

Each type below shows the content field(s) it needs. Mix freely with the shared chrome fields (time, status, quote, …) from the previous section.

Text

Plain text with WhatsApp markdown: `*bold*`, `_italic_`, `~strikethrough~` and ```` ```monospace``` ````. URLs become links; a message that is only emoji renders large with no bubble.

```
{ "from": "bot", "type": "text", "text": "We are *open* until _9pm_!", "time": "09:41" }
```

Image

Set `image` to a URL. An optional `text` becomes the caption.

```
{ "from": "bot", "type": "image",
  "image": "https://…/photo.jpg",
  "text": "Here is the look", "time": "09:41" }
```

Video

Set `video` (and ideally a `poster` image). A play button is overlaid; an optional caption is supported.

```
{ "from": "bot", "type": "video",
  "poster": "https://…/cover.jpg",
  "video": "https://…/clip.mp4", "time": "09:41" }
```

Voice note

An interactive voice bubble with a waveform you can play. `voice` may be a duration string or an object.

```
{ "from": "bot", "type": "voice",
  "voice": { "duration": "0:14" }, "time": "09:41" }
```

Document

A file bubble with a tinted icon (red for PDF, blue for DOCX).

```
{ "from": "user", "type": "document",
  "document": { "name": "Brief.pdf", "size": "240 kB", "pages": "12 pages", "ext": "PDF" },
  "time": "09:41", "status": "read" }
```

Sticker

A transparent sticker image, rendered without a bubble.

```
{ "from": "bot", "type": "sticker", "sticker": "https://…/party.png", "time": "09:41" }
```

Reply buttons

Up to three tappable quick-reply buttons under a message. Each button is a string or `{ title, reply?, icon?, selected? }`. Mark one `selected` to show it greyed, as WhatsApp does after a tap.

```
{ "from": "bot", "type": "buttons",
  "text": "Shall I lock in *Friday 3pm*?",
  "buttons": [
    { "title": "Confirm", "selected": true },
    { "title": "Pick another time" }
  ],
  "time": "09:41" }
```

List menu

A single-select menu. The bubble shows a button; tapping it opens a bottom sheet with sections and rows. Rows may be `disabled` (greyed, e.g. “Coming soon”).

```
{ "from": "bot", "type": "list",
  "list": {
    "header": "Choose a service", "body": "Pick one to continue.",
    "footer": "Bella Beauty Studio", "button": "View services",
    "sections": [
      { "title": "Popular", "rows": [
        { "id": "cut",   "title": "Cut & finish", "description": "45 min · £40" },
        { "id": "color", "title": "Full colour",  "description": "from £90" },
        { "id": "spa",   "title": "Spa (soon)",   "description": "Soon", "disabled": true }
      ] }
    ]
  },
  "time": "09:41" }
```

Poll

An interactive poll. Tap options to vote; bars animate. Set `multiple` for multi-select and optional starting `votes`.

```
{ "from": "bot", "type": "poll",
  "poll": {
    "question": "Which slot suits you?",
    "multiple": false,
    "options": [
      { "text": "Friday 3pm", "votes": 0 },
      { "text": "Saturday 11am", "votes": 0 }
    ]
  },
  "time": "09:41" }
```

Location

A map preview with a name and address. Provide a `map` image URL, or let the app draw a stylised placeholder.

```
{ "from": "bot", "type": "location",
  "location": { "name": "Bella Studio", "address": "12 King St, Bristol" },
  "time": "09:41" }
```

Contact card

A shareable contact with an avatar and a Message button.

```
{ "from": "bot", "type": "contact",
  "contact": { "name": "Front Desk", "phone": "+44 117 …", "org": "Bella Beauty" },
  "time": "09:41" }
```

Call-to-action link

A message with a button that opens a URL in a new tab.

```
{ "from": "bot", "type": "cta",
  "cta": { "text": "Add the booking to your calendar.", "display": "Add to calendar", "url": "https://…"
  "time": "09:41" }
```

Rich card

A media card with image, title, subtitle, body and its own buttons — handy for confirmations or featured content.

```
{ "from": "bot", "type": "card",
  "card": {
    "image": "https://…/room.jpg",
    "title": "Booking confirmed", "subtitle": "Fri 3 May · 3:00pm",
    "body": "We've reserved your chair. Reply to reschedule.",
    "buttons": [ { "title": "Reschedule" }, { "title": "Directions" } ]
  },
  "time": "09:41" }
```

Product

A catalogue-style product with image, name, price and an optional catalogue button.

```
{ "from": "bot", "type": "product",
  "product": { "image": "https://…/serum.jpg", "name": "Repair Serum", "price": "£28.00", "description":
  "time": "09:41" }
```

Reaction

A reaction is authored as its own message; the emoji attaches to the *previous* bubble, exactly like tapping-and-holding to react. It is not a standalone bubble.

```
{ "from": "user", "type": "reaction", "reaction": "(any emoji)" }
```

Date & system rows

Centred separators with no sender. Use `date` for the day pill and `system` for notices such as the encryption banner.

```
{ "type": "date", "text": "Today" }
{ "type": "system", "text": "Messages are end-to-end encrypted." }
```

Playback & timing

The player reveals messages one at a time. By default it paces itself: incoming (bot) messages get a short typing indicator whose length scales with the text, and a natural gap follows each message. You can override this per message.

Field	Type	Description
`delay`	number (ms)	Wait this long before the message appears, instead of the automatic gap.
`typing`	boolean	true forces a typing indicator before the message; false suppresses the automatic one (good for instant user replies).
`speed`	number (Flow)	Initial speed multiplier. Viewers can also change it live (0.5× – 2×).

System and date rows appear instantly. Reactions pop onto the previous bubble after a short beat. The progress bar under the header and the controls below the phone reflect playback position.

Forcing a longer pause

```
{ "from": "bot", "type": "text", "text": "Give me one sec…", "typing": true,  "time": "09:41" },
{ "from": "bot", "type": "text", "text": "all done",         "delay": 1500,   "time": "09:41" }
```

Theming & wallpaper

Set `theme` to `"dark"` or `"light"`. Colours are driven by CSS variables in `app/globals.css`, so the whole UI re-themes from one place. The `wallpaper` field accepts three forms:

- "default" — the subtle WhatsApp doodle pattern (tuned per theme).
- A #hex colour — a flat colour behind the bubbles.
- An image URL — any photo or texture, sized to cover.

The Flow Builder

Open `/builder` (the green pencil button, or “New conversation flow”). It has a JSON editor on the left and a live phone preview on the right; on mobile, switch between **Edit** and **Preview** tabs.

- Insert message — click any snippet (Bot text, Image, Reply buttons, List, Poll…) to append a ready-made block to the JSON.
- Live preview - valid JSON re-plays automatically a beat after you stop typing; press Replay at any time.
- Load — start from a bundled demo or a blank Flow.
- Format — pretty-print the JSON.
- Save — store the Flow in your browser; it then appears in the chats list to share.
- Export / Import — download the Flow as a .json file, or load one back in.

Validation runs continuously. Errors (bad JSON, a message with no recognisable content, an empty Flow) are listed under the editor, and the badge in the top bar shows `valid` or `errors`.

The agent workflow: brief, understand, generate

The landing page is the agent — a senior WhatsApp Business Platform developer and business analyst. Rather than generating blindly from one line, it works the way a good consultant would: it takes a brief, tells you what it understood, lets you correct it, and only then builds.

1 · Brief it

- Business name, and a short description of what the business does.
- The primary goal of the bot (take bookings, take orders, capture leads, answer questions, drive to checkout, book viewings).
- An optional tone (friendly, professional, playful, premium, minimal).
- Optional contact details — phone, website, email, opening hours, address — which the bot will weave into the conversation as a contact card, a location, and the checkout link.
- Or tap an example to pre-fill the brief and jump straight to the agent's read.

2 · See what it understands

The agent replies with a structured brief you can check before anything is built: a plain-language summary, the business type, audience and primary goal, the proposed conversation step by step, the specific WhatsApp features it will use and why, honest notes on what the platform can and can't do, and up to three questions whose answers would sharpen the bot. A line tells you whether this came from the AI agent or the offline analysis.

3 · Correct, then generate

Anything wrong or missing goes in the corrections box — answer a question, drop a constraint (“pickup only, no delivery”), or adjust the tone. **Re-analyze with my notes** folds your feedback back in; **Looks good — generate the bot** produces the conversation, saves it to your chats, and starts playing it.

What the generated conversation shows

- Detects the business category (restaurant, café, grocery, retail, beauty, fitness, clinic, real estate, hotel, education, automotive, events, or professional services).
- Uses the name and contacts you gave it, and themes everything to the business: “a restaurant that sells fruit” produces a fruit menu, with prices in the right currency (£ / $ / €).
- Demonstrates many bot functions: a welcome with quick-reply buttons, a list menu of real offerings with prices, a product or rich card, a checkout call-to-action, and a mix of voice note, location, contact card, reaction, and a receipt document.
- Is deterministic offline — the same brief yields the same demo, so your pitch is repeatable — while different briefs feel distinct.

In a hurry, **Skip — just generate** bypasses the understanding step. The Builder also keeps a quick “Generate from a business idea” box for seeding the JSON editor directly. Generation works fully offline, so there is nothing to fail in front of a client.

Optional: a sharper agent with your API key

Everything above works with built-in heuristics and no setup. If you provide your own Anthropic API key, all three agent steps — understanding your business, generating the conversation, and reviewing your edits — are handled by Claude for more bespoke, detailed results. Create a `.env.local` file in the project root:

Enable the AI agent

```
# .env.local  (optional)
ANTHROPIC_API_KEY=sk-ant-...
# optional — defaults to a current Sonnet model
ANTHROPIC_MODEL=claude-sonnet-4-6
```

Restart the dev server after adding it. If the key is missing, invalid, or the request times out, the app silently falls back to the built-in offline generator — every demo still works.

Editing with the agent — Review & refine

Open any flow in the Builder and the editor opens in **Review & refine** mode (there's a **Review | JSON** toggle if you prefer raw JSON). Review lists every message in order, each tagged `Customer`, `Bot`, or `System`, so you can rewrite the words either side says — not just the bot's.

Edit either party, get instant feedback

- Edit the text of any message directly in place — the customer's lines and the bot's replies alike. Changes flow straight back into the conversation and the live preview.
- When you finish an edit, the agent reviews it and returns a verdict — “Looks good”, a “Tip”, or a “Heads up” — with a short, specific note and an optional suggestion.
- Feedback is grounded in WhatsApp reality: it flags all-caps, bubbles that run too long, missing calls-to-action, button titles over 20 characters, and similar pitfalls.
- A Review button on each message re-runs the check on demand — handy after you add a note but leave the text as-is.

Explain your change (optional)

Each message has an *“Add a note explaining your change”* field. Anything you type there is passed to the agent as context, so its feedback responds to your intent — say `“shorter for mobile”` or `“legal wants this exact wording”` and it will take that into account. The note is saved with the message as authoring metadata and never appears in the chat itself.

As with the rest of the agent, review runs on offline heuristics by default and uses Claude when an API key is present (an `offline` tag marks heuristic feedback).

Exporting screenshots for clients

Open any chat and you’ll find two export actions — on desktop in the top bar, on mobile as floating buttons above the playback controls.

- Screenshot — captures the current phone view exactly as it appears: status bar, header, the visible bubbles and the composer. Perfect for a single hero image.
- Full chat — renders the entire conversation (every message, fully revealed) as one tall PNG, so you can send a client the whole flow at a glance.
- Both export at 2× resolution for crisp, retina-quality files, and download straight to your device.

Tip: for a single screenshot, pause or skip to the moment you want first. The full-chat export always shows the complete conversation regardless of where playback is. These images are ideal for proposals, decks, and social posts that help a prospective client picture their own bot.

Interactivity & realism

The showcase is scripted, but several elements respond to touch so a demo feels alive in a client's hands:

- List menus open a real bottom sheet; choosing a row fills the radio and dismisses it.
- Polls are tappable — votes and percentage bars update with animation.
- Voice notes play, animating the waveform and elapsed time.
- The composer works: type and send to append a genuine outgoing bubble.
- Authentic chrome — status bar, signal/battery, verified badge, ticks, quoted replies, forwarded labels, date pills, the encryption notice and the Android gesture bar.
- Grouped bubbles and tails match WhatsApp: consecutive messages from one side tuck together, only the first carries a tail.

Deployment

It's a normal Next.js app, so any Node host works. The simplest is Vercel:

Deploy to Vercel

```
# push the folder to a Git repo, then import it at vercel.com
# — or use the CLI:
npm i -g vercel
vercel
```

To self-host, run `npm run build` then `npm run start` behind your reverse proxy. Remote images (Unsplash, etc.) are already permitted in `next.config.mjs`; tighten that list for production if you prefer to allow only your own domains.

A good demo, start to finish

- Open the Builder and Load the closest bundled demo.
- Rename it, set the avatar initials/colour and the header subtitle.
- Insert and edit messages; keep the bot's wording exactly as the client will read it.
- Use typing/delay to control rhythm — a touch of waiting before a recommendation lands well.
- Save, then share the /chat link. Press play and let it tell the story.

Built with Next.js 14, TypeScript, Tailwind CSS and Framer Motion. This tool recreates the look of WhatsApp for prototyping; it is not affiliated with or endorsed by WhatsApp or Meta.